ANSIBLE BASICS TO PLAYBOOKS

Complete Student Notes

Beginner-friendly theory • Commands • YAML • Practical examples

Topics covered
What is Ansible • Use cases • Architecture • Installation on RHEL-family and Debian-family systems • Ad-hoc commands • Modules and arguments • YAML • Playbooks • Syntax checking • Execution • Practical examples

Prepared for Linux and DevOps classroom teaching

Examples use sample hostnames and IP addresses. Replace them with your actual lab details.

Contents

• 1. What is Ansible?

• 2. Why and when do we use Ansible?

• 3. Ansible architecture

• 4. Ansible installation: RHEL-family and Debian-family

• 5. Inventory and connectivity basics

• 6. What is an ad-hoc command?

• 7. Ad-hoc command syntax, modules, and arguments

• 8. Common ad-hoc command examples

• 9. What is a playbook?

• 10. What is YAML format?

• 11. Ad-hoc command vs playbook

• 12. Playbook file structure and components

• 13. Creating and running your first playbook

• 14. Playbook examples

• 15. Syntax checking and execution options

• 16. Common errors and troubleshooting

• 17. Student exercises and viva questions

1. What is Ansible?

Definition: Ansible is an IT automation tool used to configure systems, deploy applications, manage services, and automate repetitive administration tasks across multiple computers.

Malayalam / Manglish: Multiple Linux servers-il manual aayi cheyyenda installation, configuration, user management, service management poleyulla tasks automatic aayi cheyyan Ansible use cheyyunnu.

Important characteristics

• **Agentless:** For typical Linux management, Ansible connects to managed hosts using SSH; an Ansible agent usually does not need to be installed on every target.

• **Control node:** The computer where Ansible commands and playbooks are run.

• **Managed node:** The server that Ansible manages.

• **Inventory:** The hosts and groups Ansible can target.

• **Modules:** Components that perform operations on target systems.

• **YAML playbooks:** Files used to describe repeatable automation tasks.

• **Idempotency:** Many modules can maintain a desired state without unnecessary changes on repeated runs.

2. Why and when do we use Ansible?

Why use Ansible?

• To install packages on many servers.

• To create users, groups, directories, and files.

• To manage services and configuration files.

• To deploy websites and applications.

• To collect information from managed servers.

• To reduce repetitive manual work and configuration mistakes.

• To apply consistent configurations across development, testing, and production systems.

When should we use it?

Use Ansible when the same task must be repeated, when several servers need the same configuration, or when a process needs to be documented and rerun reliably. For a quick one-time check, an ad-hoc command may be enough; for a repeatable sequence of tasks, a playbook is usually better.

Example: A team needs to install Apache on 25 servers. Instead of logging in to every server and running commands separately, the administrator can target the webservers inventory group with Ansible.

3. Ansible Architecture

Administrator / Linux Trainer
Writes commands, inventory and YAML playbooks
Control Node Ansible installed here
SSH connection for Linux managed nodes
Managed Node 1 **Managed Node 2** **Managed Node 3**

Typical Linux workflow: the administrator runs Ansible on the control node; Ansible reads the inventory and task instructions; it connects to selected managed nodes over SSH, performs the required work, and reports the result.

Important: the managed node still needs suitable SSH access, permissions, and a compatible Python environment for many modules. Agentless does not mean there are no prerequisites.

4. Installing Ansible

Choose the instructions appropriate to the operating system on your control node. Package names and available Ansible versions vary by distribution and release.

A. RHEL-family systems

Examples: RHEL, Rocky Linux, AlmaLinux, and Fedora. First update package metadata and check which Ansible packages are available in your configured repositories.

```
# Check the operating system
cat /etc/os-release

# Refresh metadata (supported on many RHEL-family systems)
sudo dnf makecache

# Check package availability
dnf info ansible-core

# Install Ansible Core if available in enabled repositories
sudo dnf install ansible-core -y

# Verify
ansible --version
ansible-playbook --version
```

On some RHEL releases, repositories or subscriptions may need to be enabled before the package is available. Fedora or other releases may offer a package named ansible instead. Follow the official repository guidance for your specific distribution.

B. Debian-family systems

Examples: Debian and Ubuntu. The package available from the distribution repositories may be named ansible or ansible-core depending on release and repository configuration.

```
# Refresh package metadata
sudo apt update

# Check available package names
apt-cache policy ansible ansible-core

# Install the package available for your release
sudo apt install ansible -y

# If your repository uses the core package instead:
# sudo apt install ansible-core -y

# Verify
ansible --version
ansible-playbook --version
```

If the package is not available or the version is too old for your requirements, consult the official Ansible installation documentation for the supported method for your operating system. Avoid mixing system packages and pip installations without understanding Python environment management.

Control node vs managed node

Control node	Managed node
Install Ansible here.	Ansible normally does not need to be installed here.
Runs ansible and ansible-playbook commands.	Receives remote tasks over SSH in a typical Linux setup.
Stores inventory, configuration and playbooks.	Needs a reachable SSH service, valid account and permissions.

5. Inventory and Connectivity Basics

An inventory tells Ansible which machines to manage. It can contain hostnames, IP addresses, groups, and connection variables.

Create a sample inventory

```
# inventory.ini
[webservers]
webserver1 ansible_host=192.168.1.21 ansible_user=ansible
webserver2 ansible_host=192.168.1.22 ansible_user=ansible
```

The addresses and username above are examples. Replace them with actual lab details. Ensure SSH access works before running automation.

```
# Test connectivity from the control node
ansible -i inventory.ini webservers -m ansible.builtin.ping

# Display inventory groups
ansible-inventory -i inventory.ini --graph
```

If SSH requires a password, `-k` prompts for the SSH password. If a task requires sudo and sudo prompts for a password, `-K` prompts for the become password.

6. What is an Ad-hoc Command?

Definition: An Ansible ad-hoc command is a one-line command used to perform a quick task on one or more managed hosts without creating a playbook file.

Malayalam / Manglish: Oru quick task cheyyan vendi terminal-il direct aayi Ansible command run cheyyunnathaanu ad-hoc command. Example: server reachability check cheyyuka, hostname display cheyyuka, package install cheyyuka.

What is it used for?

• Quick connectivity checks.

• Checking hostname, uptime, disk usage, or operating-system facts.

• Creating a directory or copying a file.

• Installing a package or managing a service.

• Performing simple operations on a selected host group.

General syntax

```
ansible <host-pattern> -i <inventory-file> -m <module> -a "<arguments>"
```

Part	Meaning
ansible	Runs the ad-hoc command interface.
<host-pattern>	Host or inventory group to target.
-i	Specifies the inventory file.
-m	Selects the module.
-a	Passes module arguments as a string.

7. What are Modules and Arguments?

Module

A module is an Ansible component that performs a specific operation on a managed host.

Module	Purpose
ansible.builtin.ping	Tests Ansible connectivity and remote Python functionality; it is not an ICMP ping.
ansible.builtin.command	Runs a command without shell features by default.
ansible.builtin.shell	Runs a command through a shell; use only when shell features are needed.
ansible.builtin.copy	Copies a file or supplied content to a managed host.
ansible.builtin.file	Manages file and directory attributes or state.
ansible.builtin.dnf	Manages packages on DNF-based systems.
ansible.builtin.apt	Manages packages on Debian-family systems.
ansible.builtin.service	Manages services.
ansible.builtin.debug	Displays messages or variable values, especially in playbooks.

Arguments

Arguments are the options or values passed to a module to tell it exactly what to do. In an ad-hoc command, `-a` supplies these arguments.

```
ansible webservers -i inventory.ini \
  -m ansible.builtin.file \
  -a "path=/opt/student-data state=directory mode=0755"
```

• `path=/opt/student-data` is an argument specifying the path.

• `state=directory` requests that the path be a directory.

• `mode=0755` sets the directory permissions.

The module defines which arguments are valid. Different modules accept different arguments.

8. Common Ad-hoc Command Examples

1. Test Ansible connectivity

```
ansible -i inventory.ini webservers -m ansible.builtin.ping
```

Typical successful module result contains `ping: pong`. This is an Ansible connectivity test, not an ICMP echo request.

2. Check hostname

```
ansible -i inventory.ini webservers -m ansible.builtin.command -a "hostname"
```

3. Check uptime

```
ansible -i inventory.ini webservers -m ansible.builtin.command -a "uptime"
```

4. Check disk usage

```
ansible -i inventory.ini webservers -m ansible.builtin.command -a "df -h"
```

5. Create a directory

```
ansible -i inventory.ini webservers -b \
  -m ansible.builtin.file \
  -a "path=/opt/student-data state=directory mode=0755"
```

The `-b` option enables privilege escalation. Use it when the task requires elevated permissions.

6. Install Apache on RHEL-family managed nodes

```
ansible -i inventory.ini webservers -b \
  -m ansible.builtin.dnf \
  -a "name=httpd state=present"
```

7. Install Apache on Debian-family managed nodes

```
ansible -i inventory.ini webservers -b \
  -m ansible.builtin.apt \
  -a "name=apache2 state=present update_cache=true"
```

8. Start Apache on RHEL-family managed nodes

```
ansible -i inventory.ini webservers -b \
  -m ansible.builtin.service \
  -a "name=httpd state=started enabled=true"
```

9. Create a file

```
ansible -i inventory.ini webservers -b \
  -m ansible.builtin.copy \
  -a "content='Hello from Ansible' dest=/opt/hello.txt mode=0644"
```

Ad-hoc commands that change packages, files, or services can alter real systems. Practise in a lab and check the target group before running them.

9. What is an Ansible Playbook?

Definition: An Ansible playbook is a YAML file containing one or more plays, with tasks and settings that describe automation to perform on selected hosts.

Malayalam / Manglish: Oru sequence of tasks YAML file-il save cheythu repeat cheyyan pattunna reethiyil automate cheyyunnathaanu playbook. Example: Apache install cheyyuka, website file create cheyyuka, service start cheyyuka.

Why use a playbook?

• To combine multiple tasks in one file.

• To run tasks in a defined order.

• To reuse, review, and maintain automation.

• To apply the same workflow to multiple servers.

• To keep automation instructions in version control.

10. What is YAML Format?

YAML is a human-readable format for structured data. Ansible playbooks use indentation to show relationships between plays, tasks, modules, and arguments.

```
student:
  name: Anu
  course: Linux
  active: true
```

```
packages:
  - httpd
  - git
```

• Use spaces for indentation; do not use tabs.

• A colon separates a key from its value.

• A hyphen followed by a space starts a list item.

• Nested values must be indented consistently.

• Quote values when needed to avoid ambiguity.

• Use `ansible-playbook --syntax-check` to validate playbook syntax.

Incorrect vs correct indentation

```
# Incorrect
tasks:
- name: Install Apache
  ansible.builtin.dnf:
  name: httpd
  state: present

# Correct
tasks:
  - name: Install Apache
    ansible.builtin.dnf:
      name: httpd
      state: present
```

11. Ad-hoc Command vs Playbook

Ad-hoc command	Playbook
Usually a one-line command.	Instructions are stored in a YAML file.
Best for quick, simple tasks.	Best for multi-step, repeatable workflows.
No separate task file is required.	Requires a playbook file.
Uses -m to select a module and -a to pass arguments.	Uses YAML keys such as hosts, tasks, and module arguments.
Example: check uptime on servers.	Example: install Apache, create a page, and start the service.

12. Playbook File: Main Components

```
---
- name: Descriptive name for this play
  hosts: webservers
  become: true
  gather_facts: true

  vars:
    example_value: sample

  tasks:
    - name: Describe the task
      ansible.builtin.debug:
        msg: "This is a task"

  handlers:
    - name: Example handler
      ansible.builtin.debug:
        msg: "Handler executed"
```

Component	Purpose
---	YAML document start marker.
- name	Description of a play; the leading hyphen starts a list item.
hosts	Target host pattern or inventory group.
become	Privilege escalation setting.

Component	Purpose
gather_facts	Whether host facts are collected before tasks.
vars	Optional section for play variables; shown for learning structure only.
tasks	Ordered list of operations.
Module and arguments	The operation and its configuration.
handlers	Optional special tasks triggered by notifications.

Not every playbook must contain every section. A basic playbook may only need `hosts` and `tasks`. `vars` and `handlers` are optional and should be added when the task requires them.

This guide introduces the structure only. Variables are not covered as a detailed topic in this module.

13. Create and Run Your First Playbook

Step 1: Create a project directory

```
mkdir -p ~/ansible-lab
cd ~/ansible-lab
```

Step 2: Create inventory.ini

```
[webservers]
webserver1 ansible_host=192.168.1.21 ansible_user=ansible
```

Step 3: Create first-playbook.yml

```
nano first-playbook.yml

---
- name: First playbook
  hosts: webservers
  gather_facts: false

  tasks:
    - name: Display a welcome message
      ansible.builtin.debug:
        msg: "Welcome to Ansible Playbooks"

    - name: Check the managed host hostname
      ansible.builtin.command:
        cmd: hostname
      register: hostname_result

    - name: Display the hostname result
      ansible.builtin.debug:
        msg: "The managed host hostname was collected successfully"
```

This example demonstrates task ordering and the debug and command modules. The `register` keyword stores a task result for later use; it is included only as a structural example and is not a detailed variables lesson.

Step 4: Check syntax

```
ansible-playbook -i inventory.ini first-playbook.yml --syntax-check
```

Step 5: List selected hosts and tasks

```
ansible-playbook -i inventory.ini first-playbook.yml --list-hosts
ansible-playbook -i inventory.ini first-playbook.yml --list-tasks
```

Step 6: Execute

```
ansible-playbook -i inventory.ini first-playbook.yml
```

Ansible output normally shows each play and task, a status such as `ok` or `changed`, and a final PLAY RECAP. Exact output varies by version and environment.

14. Practical Playbook Example: Apache Website

The following example is for AlmaLinux/Rocky Linux managed hosts, where Apache is normally named `httpd`. The managed user must have appropriate sudo permissions.

```
---
- name: Install and configure Apache
  hosts: webservers
  become: true

  tasks:
    - name: Install Apache package
      ansible.builtin.dnf:
        name: httpd
        state: present

    - name: Create a custom website page
      ansible.builtin.copy:
        dest: /var/www/html/index.html
        content: |
          <html>
          <head>
            <title>Student Lab</title>
          </head>
          <body>
            <h1>Welcome to Ansible Automation</h1>
            <p>This page was deployed by a playbook.</p>
          </body>
          </html>
        owner: root
        group: root
        mode: '0644'

    - name: Ensure Apache is running
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: true
```

Run the playbook

```
ansible-playbook -i inventory.ini apache-site.yml -K
```

Verify on the managed host

```
systemctl status httpd
curl http://localhost/
```

Save the YAML above as `apache-site.yml` before running it. Ensure the inventory file exists in the current directory or specify its full path.

Ubuntu variation: use the `apt` module with package `apache2`, the service name `apache2`, and the appropriate web root for the target system. Do not use the RHEL-family example unchanged on Ubuntu.

15. Syntax Checking and Useful Execution Options

Command	Purpose
ansible-playbook -i inventory.ini file.yml --syntax-check	Check syntax without executing tasks.
ansible-playbook -i inventory.ini file.yml --list-hosts	List target hosts.
ansible-playbook -i inventory.ini file.yml --list-tasks	List tasks.
ansible-playbook -i inventory.ini file.yml --check	Attempt check mode where supported.
ansible-playbook -i inventory.ini file.yml --diff	Show supported file/configuration differences.
ansible-playbook -i inventory.ini file.yml --limit webserver1	Run against a limited host selection.
ansible-playbook -i inventory.ini file.yml -K	Prompt for become password if needed.
ansible-playbook -i inventory.ini file.yml -v	Show additional details.

Check mode is not a perfect simulation of every task. Some modules do not support it fully, and dependent tasks may behave differently from a real run.

16. Common Errors and Troubleshooting

Problem	Likely reason	First check
Inventory parsing error	Incorrect inventory syntax or wrong file	Check file path and INI group syntax.
No hosts matched	Group name mismatch	Compare inventory group with hosts:.
UNREACHABLE	SSH or network issue	Test SSH manually and verify IP/routing.
Permission denied	Wrong SSH user/key or insufficient permissions	Check credentials and sudo access.
Missing sudo password	Privilege escalation needs a password	Use -K if appropriate.
YAML syntax error	Incorrect indentation or syntax	Run --syntax-check and inspect spaces.
Package not found	Wrong OS/package name or repo	Check distribution and package availability.
Module arguments error	Incorrect or unsupported argument	Read module documentation for installed version.

17. Student Practice Exercises

• Write an ad-hoc command to check the hostname of all webservers.

• Write an ad-hoc command to display disk usage with df -h.

• Create a directory named /opt/linux-lab on all managed hosts.

• Install Apache on an AlmaLinux/Rocky Linux host using an ad-hoc command.

• Create a playbook that displays a welcome message.

• Create a playbook that checks the hostname of managed hosts.

• Create a playbook that installs Apache and starts the service.

• Add a task to copy a custom HTML page to the web root.

• Run --syntax-check before executing each playbook.

• Explain the difference between an ad-hoc command and a playbook.

18. Viva Questions and Answers

What is Ansible?
An IT automation tool for configuration management, deployment, and repetitive administration tasks.

What is a control node?
The machine where Ansible commands and playbooks are run.

What is a managed node?
A target machine that Ansible manages.

What is inventory?
A list or structure of hosts and groups that Ansible can target.

What is an ad-hoc command?
A one-line command for a quick task without creating a playbook file.

What is a module?
A component that performs a particular operation.

What are arguments?
Values/options passed to a module to define the operation.

What does -m do?
Selects a module for an ad-hoc command.

What does -a do?
Passes arguments to the selected module.

What is a playbook?
A YAML file containing plays and automation tasks.

What is YAML?
A human-readable format for structured data.

What does hosts specify?
The hosts or inventory group targeted by a play.

What does become: true do?
Enables privilege escalation when required.

What does --syntax-check do?
Checks playbook syntax without executing the tasks.

What is the Ansible ping module checking?
Ansible connectivity and remote Python functionality, not ICMP ping.

Final Quick Revision

• Ansible automates configuration and administration tasks across systems.

• Ansible is commonly agentless for Linux and uses SSH to connect to managed nodes.

• Inventory identifies hosts and groups.

• An ad-hoc command is useful for quick tasks.

• A module performs an operation; arguments configure that operation.

• A playbook is a YAML file containing plays and tasks.

• YAML indentation matters; use spaces rather than tabs.

• Use ansible-playbook --syntax-check before executing a new playbook.

• Validate results on the managed node and troubleshoot SSH, permissions, package names, and YAML errors.

Trainer suggestion: Teach the concepts first, demonstrate each command on the control node, ask students to predict the output, then let them repeat the lab and explain each line in their own words.